REVERSE ENGINEERING WRITEUP

vodeff's FindLicenseKey

Key-generation crackme analysis - Linux x86-64

Challenge author	vodeff
Category	Crackme / keygen
Architecture	x86-64
Platform	GNU/Linux
Binary type	ELF 64-bit PIE, dynamically linked, stripped
Constraint	Generate a valid key; no patching

Prepared by Cenzer0

This writeup explains the complete reasoning process and does not patch the supplied executable.

1. Challenge Objective

The program is executed as `./findlicensekey username`. It then asks for a license key. The goal is to understand how the executable derives the expected key from the supplied username and to reproduce that calculation in a separate key generator. Patching the comparison or bypassing the check is explicitly outside the intended solution.

2. Initial Triage

I first identified the executable format and basic properties with the standard `file` utility:

```
file findlicensekey

findlicensekey: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
for GNU/Linux 4.4.0, stripped
```

The important observations were:

- The target is a Linux ELF binary, not a native macOS executable.
- It is 64-bit x86-64 and position-independent (PIE).
- It is dynamically linked and stripped, so most original symbol names are absent.
- Because the executable is small, a focused static analysis is sufficient.

3. Runtime Behaviour

The command-line argument is used as the username. When a username is supplied, the program prompts for a license key and reports whether the entered value is correct:

```
./findlicensekey AAAAAAAAAAAAAAAAAAAAAAA
Enter license key to continue:
```

Running the Linux executable on macOS directly results in an `exec format error`; therefore, runtime validation must be performed on Linux, inside a compatible virtual machine, or in an amd64 Linux container.

4. Static Analysis Strategy

Since the binary is stripped, I approached it by locating the input, generation, and comparison paths rather than relying on function names.

1. Inspect printable strings to identify prompts and success/failure messages.
2. Locate references to those strings in the disassembly.
3. Follow the control flow around the license input and final comparison.
4. Identify the loop that constructs the expected 24-byte key.
5. Translate the loop into readable pseudocode and reproduce it independently.

The relevant behaviour can be summarized as follows:

```
username = argv[1]
expected_key = generate_key(username)
entered_key = read_from_stdin()

if strcmp(entered_key, expected_key) == 0:
    print("Key validated")
else:
    print("Invalid key")
```

5. Recovering the Character Table

The key-generation loop indexes into a fixed 62-character lookup table embedded in the executable:

```
QAZPLWSXOKMEYDCIJNRFVUHBTGqpalzmwoeirutyskdjfhgxncbv1750284369
```

The table contains exactly 62 characters. This size is significant because every generated character is selected using a modulo-62 operation.

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Q	A	Z	P	L	W	S	X

The complete table is used by index; the small sample above illustrates the index-to-character relationship.

6. Reconstructing the Algorithm

The program generates exactly 24 output characters. For each position `i`, it reads the corresponding username byte, adds the position index, reduces the result modulo 62, and uses that value as an index into the lookup table.

```
for i from 0 to 23:
    table_index = (username[i] + i) % 62
    output[i] = alphabet[table_index]
output[24] = '\0'
```

Equivalent C-like pseudocode:

```
void generate_key(const unsigned char *username, char *output) {
    const char *alphabet =
        "QAZPLWSXOKMEYDCIJNRFVUHBTG"
        "qpalzmwoeirutyskdjfhgxncbv"
        "1750284369";

    for (int i = 0; i < 24; i++) {
        output[i] = alphabet[(username[i] + i) % 62];
    }
    output[24] = '\0';
}
```

A practical detail is that the routine always processes 24 username bytes. Using a username of exactly 24 ASCII characters avoids dependence on bytes beyond a shorter string terminator and makes the result deterministic.

7. Manual Worked Example

I used 24 uppercase `A` characters as a controlled username:

```
AAAAAAAAAAAAAAAAAAAAAAAA
```

ASCII `A` is decimal 65. The first positions are therefore calculated as:

i	Username byte	byte + i	Modulo 62	Output
0	65	65	3	P
1	65	66	4	L
2	65	67	5	W
3	65	68	6	S
4	65	69	7	X
5	65	70	8	O

Applying the same operation to all 24 positions produces:

```
PLWSXOKMEYDCIJNRFVUHBTGq
```

8. Independent Key Generator

The following Python implementation reproduces the recovered algorithm. It requires an exact 24-character ASCII username so that its behaviour remains deterministic and easy to verify.

```
#!/usr/bin/env python3
import sys

ALPHABET = (
    "QAZPLWSXOKMEYDCIJNRFVUHBTG"
    "qpalzmwoeirutyskdjfhgxncbv"
    "1750284369"
)
LENGTH = 24


def generate_key(username: str) -> str:
    raw = username.encode("ascii")
    if len(raw) != LENGTH:
        raise ValueError("username must be exactly 24 ASCII characters")

    return "".join(
        ALPHABET[(byte + index) % len(ALPHABET)]
        for index, byte in enumerate(raw)
    )


if __name__ == "__main__":
    if len(sys.argv) != 2:
        raise SystemExit(f"usage: {sys.argv[0]} <24-char-username>")
    print(generate_key(sys.argv[1]))
```

9. Validation

The generated key was tested against the original unmodified executable in a compatible Linux x86-64 environment:

```
$ python3 keygen.py AAAAAAAAAAAAAAAAAAAAAAAA
PLWSXOKMEYDCIJNRFVUHBTGq

$ ./findlicensekey AAAAAAAAAAAAAAAAAAAAAAAA
Enter license key to continue:
PLWSXOKMEYDCIJNRFVUHBTGq
Key validated
```

This confirms that the lookup table, loop length, index arithmetic, and final output reconstruction all match the original program. No bytes in the crackme were modified, and no control-flow check was bypassed.

10. Why the Solution Works

- Every key character depends only on one username byte and its position.
- The fixed lookup table contains 62 characters, matching the modulo divisor.
- The output length is fixed at 24 characters.
- The verifier performs a direct string comparison against the generated result.

- Therefore, recreating the generator is enough to produce valid keys for chosen 24-byte usernames.

11. Notes on Portability and Testing

The supplied target is an ELF Linux x86-64 executable. On macOS, it cannot be launched directly. Valid testing options include:

- An x86-64 Linux machine or virtual machine.
- An amd64 Linux Docker or Podman container.
- An emulated Linux environment capable of executing x86-64 ELF binaries.

```
docker run --rm -it --platform linux/amd64 \
  -v "$PWD:/work" -w /work ubuntu:24.04 \
  ./findlicensekey AAAAAAAAAAAAAAAAAAAAAAAA
```

The container command is only a runtime compatibility method; it does not alter or patch the challenge executable.

12. Conclusion

The core of FindLicenseKey is a compact positional substitution algorithm. The program maps each of 24 username bytes through a 62-character table after adding the current index. Recovering that table and arithmetic was sufficient to create a valid independent key generator. The final result was confirmed against the original binary, satisfying the challenge requirement without patching.

Appendix: Reproduction Summary

```
Username: AAAAAAAAAAAAAAAAAAAAAAAA
Key:      PLWSXOKMEYDCIJNRFVUHBTGq
Result:   Key validated
```